

When a `return` Is Required, and When It Isn't

Handlers, guards, callbacks and arrows: what return actually does, and how to spot the missing one

October 2026

What `return` does. `return` ends the function right there. Code below it does not run. That is its whole job in a handler. It does **not** send anything to the client: `res.json(...)`, `res.status(...).end()` and `next(...)` do the sending. Express ignores whatever a handler returns.

The one question. *After this line, is there any code that could still run and respond, save, or call something a second time?* If yes, the line needs a `return`. If nothing can run after it, a `return` changes nothing.

1. Handlers: guard clauses need it, the last line does not

Program A, `getTotal`:

Line	Code	return needed?	Why
37	<code>if (!claims) return res.status(401).json(...)</code>	yes	Without it, execution falls through to line 39 and carries on as if authenticated
42	<code>return res.json(hit.value);</code>	yes	Without it, line 45 starts a DB query and line 50 sends a second response
46	<code>if (...) return res.status(404).json(...)</code>	yes	Without it, line 48 runs after the 404 was sent
50	<code>res.json(totals);</code>	no	Last statement; nothing follows it

```
// x missing return: both responses are sent
if (!user) res.status(404).json({ error: "no user" });
res.json(user);           // runs too -> "Cannot set headers after they are sent"

// ✓
if (!user) return res.status(404).json({ error: "no user" });
res.json(user);
```

Reading rule. A response call inside an `if`, with more code below the `if`, must be `returned` (or followed by an explicit `return;`). A response call on the last line does not need it.

Both spellings are correct on the last line. Many people write `return res.json(...)` everywhere so they can never forget. A review comment about a missing `return` on a final statement is a nit at most.

2. When a missing `return` on the last line DOES matter

The last statement of the *function* is not always the last thing that can run.

Situation	Why a missing <code>return</code> bites
Code after the enclosing <code>if/else</code> or <code>try</code> block	The "last line" of the branch is not the last line of the function
A <code>finally</code> block	It runs after the <code>try</code> body whether or not it returned

Situation	Why a missing <code>return</code> bites
A callback that is itself inside another handler	The outer function continues after the inner call is queued
A loop	Without <code>return</code> , the loop moves on to the next element and responds again

```
// x the 404 branch ends on its last line, but the function does not
if (!user) {
  res.status(404).json({ error: "no user" }); // last line of the BRANCH
}
res.json(user); // still runs

// x loop: responds on every iteration
for (const o of orders) { if (o.bad) res.status(400).end(); }
```

3. Functions that must return a value

When the caller uses the result, a missing `return` hands back `undefined`.

Code	Result
<code>function double(x) { x * 2 }</code>	<code>undefined</code> (expression computed, thrown away)
<code>function double(x) { return x * 2 }</code>	the number
<code>async function load() { await db.query(...) }</code>	<code>Promise<void></code> ; the rows are lost
<code>async function load() { return db.query(...) }</code>	<code>Promise<rows></code>

```
// x caller gets undefined, then undefined.length throws
async function loadOrders(id: string): Promise<Order[]> {
  const rows = await db.query(sql, [id]); // computed, never returned
}
```

TypeScript catches this one when the return type is declared (`Promise<Order[]>` with no `return` is a compile error). It does **not** catch it when the return type is left to inference, because `void` is a legal inferred type.

4. Arrow functions: the braces decide

Form	Returns	Notes
<code>x => x * 2</code>	<code>x * 2</code>	Expression body: <code>return</code> is implicit
<code>x => { x * 2 }</code>	<code>undefined</code>	Braces make it a block: needs explicit <code>return</code>
<code>x => ({ a: x })</code>	the object	Parentheses so <code>{</code> is read as an object, not a block
<code>x => { a: x }</code>	<code>undefined</code>	<code>a:</code> is read as a label, not a property

Where it bites. `arr.map(x => { x * 2 })` gives an array of `undefined`. `arr.filter(x => { x > 0 })` gives an empty array (every result is falsy). In review, any callback with braces and no `return` is a finding unless it is `forEach` or similar and returns nothing on purpose.

5. Callbacks: `return` leaves the callback, not the outer function

```
function f(items: Item[]) {
  items.forEach((it) => {
    if (it.bad) return;           // only skips this item, like `continue`
    process(it);
  });
  // f continues here regardless
}
```

A `return` inside `forEach`, `map`, `then`, or any callback ends **that callback**. To leave the outer function, use a `for...of` loop (a `return` there does leave `f`), or collect the result and return it after.

```
// x intended to stop the handler on the first bad item
orders.forEach((o) => { if (o.bad) return res.status(400).end(); });
res.json(orders);           // still runs; response sent twice

// ✓
for (const o of orders) { if (o.bad) return res.status(400).end(); }
res.json(orders);
```

6. `return` and Promises: `return await` versus `return p`

Code (inside <code>try</code>)	Rejection caught by this <code>catch</code> ?
<code>return db.query(...)</code>	no : the Promise leaves the <code>try</code> still pending; the <code>catch</code> never sees a rejection
<code>return await db.query(...)</code>	yes : the <code>await</code> is inside the <code>try</code>
<code>await db.query(...); return x;</code>	yes

Outside a `try`, `return p` and `return await p` behave the same for the caller. `return await` inside `try` is a **looks wrong, is fine** item.

7. Express error handling: `return next(err)`

```
app.get("/a", (req, res, next) => {
  if (bad) return next(new Error("bad")); // hand off, then stop
  res.json(ok);                          // would run without the return
});
```

`next(err)` does not stop the function. Without `return`, the handler hands the error off and then sends a normal response as well. Same rule as section 1.

8. The scan you run on every handler

Step	Question	Finding if...
1	Find every <code>res.json</code> , <code>res.status(...)</code> , <code>res.send</code> , <code>res.end</code> , <code>next(</code>	

Step	Question	Finding if...
2	Is it inside an <code>if</code> , loop, <code>catch</code> or callback?	
3	Is there code that can still run after it, in the same function?	yes, and no <code>return</code>
4	Is the <code>return</code> inside a callback instead of the handler?	it only leaves the callback
5	Does a function whose result is used have a <code>return</code> ?	none: caller gets <code>undefined</code>
6	Arrow with <code>{ }</code> : has a <code>return</code> ?	none: result is <code>undefined</code>
7	<code>return p</code> inside <code>try</code> : should it be <code>return await p</code> ?	yes, if the <code>catch</code> should see the rejection

Say it on the Loom. “Line 46 returns after the 404, so execution stops there. Line 50 is the last statement and has no `return`, but nothing runs after it, so that is fine. If line 37 lacked its `return`, an unauthenticated request would fall through and be served.”

One-line summary. `return` stops the function. It is required where code could still run after a response; it is optional where the response is the last thing the function does.